

Technical Design Doc: AccessHub

Internal Access Management Tool · Engineering Design Document

Status: **Draft for Review** · Owner: Platform Engineering · Version 0.9

For reviewers: This is the design the team intends to build. It is intentionally a draft — some areas are well specified and some are not. Read it as you would a real design doc and look for security-relevant decisions, assumptions, and gaps.

1. Overview & Purpose

AccessHub is an internal web application that lets employees request access to company resources (databases, dashboards, cloud accounts, shared drives, etc.) and lets managers approve or deny those requests. Today this happens over email and chat, which is slow, untracked, and error-prone. AccessHub centralizes requests, approvals, and an audit trail in one place.

1.1 Goals

- Allow any authenticated employee to request access to a resource.
- Allow managers to approve or deny requests.
- Keep a record of who requested what, when, and who approved it.
- Enforce role-based permissions so users can only do what their role allows.

1.2 Non-Goals

- AccessHub does not *provision* the access itself (it records the decision; a separate downstream system grants it).
- No single-sign-on (SSO) integration in v1.
- No mobile app in v1.

2. Users & Roles

There are exactly three roles. Every user is assigned one role at account creation.

Role	Privilege	Can do
Analyst	Lowest	Submit access requests; view the status of their own requests.
Engineer	Middle	Everything an Analyst can do, plus view all requests across the org (read-only).
Manager	Highest	Everything an Engineer can do, plus approve/deny any request, and create/edit/deactivate user accounts.

Role is stored as a string field on the user record (`role = "analyst" | "engineer" | "manager"`). The role is also written into the user's session token at login so the frontend can show/hide buttons.

3. High-Level Architecture

AccessHub is a standard three-tier web application.

Tier	Technology	Responsibility
Frontend	React SPA, served from a CDN	Renders UI; calls the API over HTTPS; stores the session token in browser localStorage.
Backend API	Node.js / Express REST API, two load-balanced containers	Authentication, authorization, business logic, audit logging.

Tier	Technology	Responsibility
Database	PostgreSQL (single primary instance)	Stores users, requests, approvals, and audit log.

The frontend and API communicate over HTTPS. The API connects to the database over the internal network using a single shared database username and password stored in an environment variable on the container.

4. Authentication

v1 uses **basic username/password authentication**. SSO is deferred to a later version.

- Users log in with a **username** (their email) and a **password** on the login page.
- Passwords are stored in the `users` table, hashed with **MD5** before storage. (Chosen because it's fast and the team is already familiar with it.)
- On successful login the API issues a session token — a **JWT** signed by the server — containing the user's id, username, and role. The token does not expire during a session.
- The frontend stores the token in `localStorage` and sends it in the `Authorization` header on every API call.
- There is **no multi-factor authentication** in v1.
- Password rules: minimum 6 characters. No other complexity or rotation requirements are defined yet.
- Failed logins: the endpoint returns “user not found” if the username doesn't exist and “wrong password” if the password is incorrect, to help users troubleshoot. There is no limit on login attempts.
- Password reset: a user emails the IT helpdesk, who sets a new temporary password for them.

5. Authorization

Authorization is role-based. The rules:

- The frontend reads the `role` claim from the JWT and hides buttons the user shouldn't use (e.g. “Approve” is only shown to managers).
- The backend checks the role on **sensitive** endpoints. Specifically, `POST /requests/:id/approve` and `/deny` verify the caller's role is manager.
- The “view all requests” endpoint (`GET /requests`) is available to engineers and managers. Analysts calling it are expected to only see their own requests — *the frontend filters the list to the current user*.
- The user-management endpoints (`POST /users`, `PATCH /users/:id`) are intended for managers only.

6. Core Workflows

6.1 Submitting a request

1. An analyst fills in a form: target resource, reason, and desired duration.
2. The frontend sends `POST /requests` with the form data and the user's JWT.
3. The API creates a row in `requests` with status `PENDING` and the requester's user id (taken from the request body field `requesterId` sent by the frontend).
4. The request appears in the manager's queue.

6.2 Approving / denying a request

1. A manager views the pending queue and clicks Approve or Deny.
2. The frontend sends `POST /requests/:id/approve` (or `/deny`).
3. The API checks the caller is a manager, updates the request status, and records the approving manager's id and a timestamp.
4. A downstream provisioning system (out of scope) reads approved requests and grants the actual access.

7. Data Model

Table	Key fields
users	id, username (email), password_md5, role, is_active, created_at
requests	id, requester_id, resource, reason, duration, status (PENDING/APPROVED/DENIED), created_at
approvals	id, request_id, decided_by (user id), decision, decided_at
audit_log	id, actor_id, action, target, timestamp

Database queries are built by concatenating user-supplied values into SQL strings in a few of the older endpoints; the team plans to migrate these to parameterized queries “later.” All four tables live in the same database accessed by the single application DB user, which has full read/write privileges.

8. API Endpoints (summary)

Method & path	Purpose	Who should call it
POST /login	Authenticate, return JWT	Anyone
POST /requests	Create a request	Any logged-in user
GET /requests	List requests	Engineers, Managers (Analysts see own)
GET /requests/:id	View one request	Any logged-in user
POST /requests/:id/approve	Approve a request	Managers only
POST /requests/:id/deny	Deny a request	Managers only
POST /users	Create a user (set role)	Managers only
PATCH /users/:id	Edit a user (incl. role)	Managers only

9. Logging & Auditing

- Approvals and denials are written to the `audit_log` table.
- Logins, request creation, and user-management actions are **not** currently logged.
- Application logs (including full request bodies) are written to a file on each container and rotated weekly. These logs are readable by anyone with container access.

10. Infrastructure & Deployment (AWS)

AccessHub runs entirely in AWS, in a **single region (us-east-1)** with no multi-region failover in v1. The table below maps each tier to the AWS services it uses — use it to draw your architecture diagram.

Layer	AWS service	Configuration notes
DNS	Route 53	Public hosted zone resolves <code>accesshub.example.com</code> to the CloudFront distribution.
CDN / Frontend	CloudFront + S3	The React SPA's static files are stored in an S3 bucket and served via a CloudFront distribution. The S3 bucket has public read enabled for simplicity (no Origin Access Control).
Edge / Routing	Application Load Balancer (ALB)	Public-facing ALB in the public subnets. TLS (ACM certificate) terminates here; traffic from the ALB to the API tasks is plain HTTP inside the VPC.

Layer	AWS service	Configuration notes
API compute	ECS Fargate	Two Fargate tasks (Node.js/Express container image pulled from ECR) running in the public subnets with public IPs, behind the ALB. Auto-scaling is not configured.
Database	RDS PostgreSQL	Single-AZ db.t3.medium instance in a private subnet. Storage encryption is not enabled. Accessed with one shared DB username/password.
Secrets	Plain ECS env vars	DB password and JWT signing key are set as plaintext environment variables in the ECS task definition. AWS Secrets Manager / SSM Parameter Store are not used. The JWT key is "accesshub-secret", identical across dev/staging/prod.
Backups	S3 bucket	Nightly pg_dump written to an S3 bucket. Bucket default encryption is off and there is no bucket policy restricting access beyond account default.
Logs	CloudWatch Logs	Container stdout/stderr (including full request bodies) streamed to a CloudWatch log group. Log group retention is set to Never Expire ; no field redaction.
Identity	IAM	The ECS task role has the broad AmazonS3FullAccess and AmazonRDSFullAccess managed policies attached for convenience.

10.1 Networking

- Single VPC (10.0.0.0/16) with two public subnets and two private subnets across two Availability Zones.
- The ALB security group allows inbound 443 and 80 from 0.0.0.0/0.
- The ECS task security group allows inbound from the ALB security group, and also allows inbound SSH-style debugging port 22/9229 from 0.0.0.0/0 (left over from development).
- The RDS security group allows inbound 5432 from the ECS task security group *and* from the developers' office IP range.
- There is **no AWS WAF** in front of CloudFront or the ALB, and **no rate limiting** anywhere.

10.2 CI/CD

- GitHub Actions builds the container image, pushes to ECR, and runs `aws ecs update-service` to deploy.
- Deploy credentials are a long-lived IAM access key stored as a GitHub repository secret (no OIDC federation, no rotation).

11. Open Questions / Known Gaps

- Should we add account lockout after repeated failed logins?
- Do we need an expiry on session tokens?
- Is MD5 acceptable for password storage, or should we revisit?
- How do we handle a manager approving their own request?
- Should analysts ever be able to call `GET /requests` directly?

End of design doc. Reviewers: the “Open Questions” above are the team’s own — your job is to find the threats they *haven’t* thought of too.